O guia especifica R$ 2.000,00, o Inquisit script define startAmount = 2000, mas o protótipo no Manus exibe R$ 2.500,00. Isso precisa ser padronizado antes da coleta — o valor afeta a percepção de risco e é variável de protocolo registrada no CEP.

seção 6.2 usa letras A/B/C/D (nomenclatura de Bechara et al., 1994), mas o script Inquisit usa "baralho 1/2/3/4". Para o participante, os rótulos na tela serão numéricos. Isso não é erro metodológico grave, mas deve ser explicitado no método da dissertação para não confundir o leitor ao comparar com a literatura.

3. Moeda exibida ao participante

O script exibe $ (cifrão americano) nos feedbacks de ganho/perda: "Você ganhou $<% values.gain%>". Deveria ser R$ para manter a validade ecológica do instrumento com amostra brasileira. Isso é editável na seção <text gain> e <text loss>.

4. Randomização de grupos

O script implementa 4 grupos de ordem de apresentação dos baralhos (deckOrderGroup 1–4) — isso está correto e alinhado com o protocolo padrão de Bechara. Bom. Certifique-se de registrar isso no método como controle da posição dos estímulos.

Texto original da instrução ao participante:

"Nesta tela, você verá quatro baralhos de cartas: A, B, C e D. Seu objetivo é ganhar o máximo de dinheiro possível e evitar perder o máximo possível. Você começa com um empréstimo de R$ 2.000,00. Alguns baralhos são melhores que outros. Continue jogando até o final."

Problemas identificados:

"evitar perder o máximo possível" é ambíguo — pode ser lido como "evite ao máximo perder" ou "evite perder o valor máximo". O script Inquisit em inglês diz simplesmente "maximize your earnings". Simplifique.

"Você começa com um empréstimo" — a palavra "empréstimo" introduz um frame de dívida que pode interferir na tomada de decisão. O protocolo original usa simplesmente "loan" de forma neutra; em português, "saldo inicial" é mais adequado.

O rótulo "A, B, C e D" conflita com o script que usa numeração.

Versão revisada sugerida:

"Nesta tarefa, você verá quatro baralhos de cartas numerados de 1 a 4. A cada rodada, escolha um baralho clicando sobre ele. Cada escolha resultará em um ganho ou uma perda de dinheiro. Você inicia com um saldo de R$ 2.000,00 e deve tentar maximizar seus ganhos ao longo das 100 jogadas. Alguns baralhos são mais vantajosos que outros."

Tabela 6.2 — problema de precisão:

A coluna "Resultado" usa "Desvantajoso/Vantajoso" sem definição operacional no texto. Em uma dissertação, essa tabela deve ter uma nota de rodapé especificando o critério: resultado líquido esperado ao longo de 10 seleções por baralho, conforme Bechara et al. (1994).

✅ O que está correto e pode ser mantido

A estrutura técnica do Inquisit está fiel ao protocolo de Bechara et al. (1994). As listas de perda (lossDeck1 a lossDeck4) implementam corretamente o esquema sem reposição, que é o padrão mais usado atualmente. A randomização de posição dos baralhos entre participantes está adequada.

Instruções

Nesta tarefa, você joga um jogo de "azar".
Você precisa escolher um dos 4 botões (A, B, C ou D) com o mouse ou click.

A cada vez, você pode ganhar algum dinheiro, mas às vezes também pode ter que pagar uma taxa para o banco.
Após cada tentativa, você precisa coletar seu dinheiro, o que ajustará seu montante de dinheiro.

Você começa com um empréstimo de $2000, mostrado no topo da tela.

Existem 100 tentativas (leva cerca de 5 minutos).

Continue até parar e veja quanto você consegue ganhar além do empréstimo de $2000.

A

A

B

B

C

C

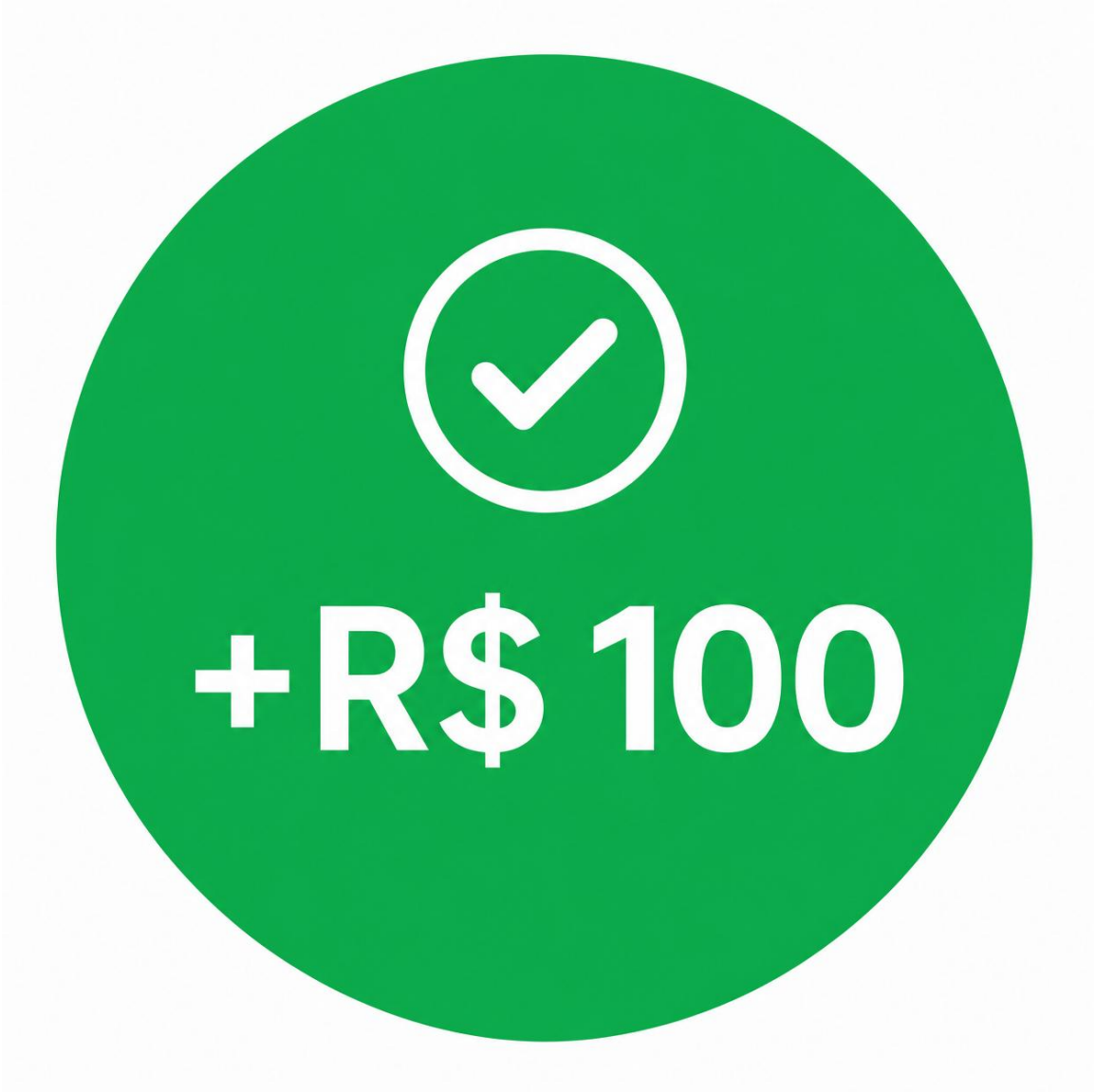
D

ESCOLHA UM BARALHO

COLETAR

-R$ 50

-R$ 250


+R$ 100

6. Guia de Implementação: Iowa Gambling Task (IGT)

O Iowa Gambling Task é uma medida comportamental de tomada de decisão sob risco e ambiguidade.

6.1 Instruções para o Participante

"Nesta tela, você verá quatro baralhos de cartas: A, B, C e D. Seu objetivo é ganhar o máximo de dinheiro possível e evitar perder o máximo possível. Você começa com um empréstimo de R$ 2.000,00. Alguns baralhos são melhores que outros. Continue jogando até o final."

6.2 Estrutura Técnica

Baralho	Recompensa/Punição	Resultado
A	Alta / Frequente e Alta	Desvantajoso
B	Alta / Infrequente e V. Alta	Desvantajoso
C	Baixa / Frequente e Baixa	Vantajoso
D	Baixa / Infrequente e Baixa	Vantajoso

7. Referências Acadêmicas

- BECK, A. T. et al. An inventory for measuring clinical anxiety: psychometric properties. J Consult Clin Psychol, 1988.
- MCELROY, E.; SHEVLIN, M. The development and initial validation of the CSS. J Anxiety Disord, 2014.
- SCHWARZER, R.; JERUSALEM, M. Generalized Self-Efficacy scale. In: Measures in health psychology, 1995.
- BECHARA, A. et al. Insensitivity to future consequences following damage to human prefrontal cortex. Cognition, 1994.
- SILVA, F. G. et al. Cross-cultural adaptation of the CSS for Brazilian Portuguese. Trends Psychiatry Psychother, 2016.

//
//Portuguese instructions for iowagamblingtask.iqjs
//

NOTE: instructions can be changed here:

<item instructions>
/ 1 = "Nesta tarefa, você será solicitado(a) a selecionar uma carta de um dos quatro baralhos apresentados acima. Você pode selecionar ao <%expressions.buttonInstruct1%>.

Em cada carta, você pode ganhar dinheiro, mas também pode perder. Alguns baralhos serão mais rentáveis do que outros. Tente escolher as cartas dos baralhos mais rentáveis, pois assim o seu ganho total será o maior possível.

Você receberá <% block.igt.trialsCount%> chances de selecionar uma carta do baralho que você acreditar ser o mais rentável. Seu ganho total e o número de cartas selecionadas serão apresentados na tela do computador.

Você inicia com $<%values.currentTotal%>. <%expressions.buttonInstruct2%> \"Iniciar\" para começar a tarefa."
/ 2 = "Selecionar uma carta."
</item>

<text cardsSelected>
/ items = ("Cartas selecionadas: <% values.cardsSelected %> de <% block.igt.trialsCount%>")
/ position = (50%, 10%)
/ fontStyle = ("Arial", 2%)
/ erase = false
</text>

<text start>
/ items = ("Iniciar")
/ position = (50%, 90%)
/ fontStyle = ("Arial", 3%, false, false, true, false, 5, 0)
/ txColor = blue
</text>

<text continue>
/ items = ("Continuar")
/ position = (50%, 65%)
/ fontStyle = ("Arial", 3%, false, false, true, false, 5, 0)
/ txColor = blue
</text>

<text gain>
/ items = ("Você ganhou $<% values.gain%>")
/ position = (20%, 51%)
/ vAlign = top
/ txColor = green
</text>

<text loss>
/ items = ("Você perdeu $<% values.loss%>")
/ position = (20%, 55%)
/ vAlign = top
/ txColor = red
</text>

```
<text previousTotal>
/ items = ("   Total anterior: $<% values.previousTotal %>   ")
/ position = (50%, 18%)
/ erase = false
</text>

<text currentTotal>
/ items = ("   Total atual: $<% values.currentTotal %>   ")
/ position = (50%, 22%)
/ erase = false
</text>

<text igt>
/ items = ("Iowa Gambling Test")
/ position = (50%, 10%)
/ fontStyle = ("Arial", 4%)
/ erase = false
</text>

<text deck1Label>
/ items = ("baralho 1")
/ position = (20%,51%)
/ vAlign = top
/ erase = false
</text>

<text deck2Label>
/ items = ("baralho 2")
/ position = (40%,51%)
/ vAlign = top
/ erase = false
</text>

<text deck3Label>
/ items = ("baralho 3")
/ position = (60%,51%)
/ vAlign = top
/ erase = false
</text>

<text deck4Label>
/ items = ("baralho 4")
/ position = (80%,51%)
```

/ vAlign = top
/ erase = false
</text>

****
general instruction expressions: adjust the instruction text depending on device used to run script
****
<expressions>
/ buttonInstruct1 = if (computer.touch && !computer.hasKeyboard) {"clicar na carta";} else {"clicar na carta";}
/ buttonInstruct2 = if (computer.touch && !computer.hasKeyboard) {"Pressione";} else {"Clique em";}
</expressions>

****
****

END OF FILE

<userManual>

IOWA GAMBLING TASK (IGT) - Helper Script

Last Updated: 07-26-2023 by K. Borchert (katjab@millisecond.com), Millisecond

Script Copyright © 07-26-2023 Millisecond Software, LLC

BACKGROUND INFO

This sample script implements the Iowa Gambling Task (IGT). The IGT task assesses risk learning

by utilizing rewards and punishment in an uncertain simulated real-life decision making context

The implemented procedure is based on:

REFERENCES

Bechara A., Damasio A. R., Damasio H., Anderson S. W. (1994).
Insensitivity to future consequences following damage to human prefrontal cortex.
Cognition, 50, 7-15 .

TASK DESCRIPTION

Participant are presented 4 decks of cards and are asked to select a card from one of the 4 decks with their mouse.
By turning over the cards, participants can win money but also lose money.
The four different decks can be categorized as advantageous (overall more wins than losses)
or disadvantageous (overall more losses than wins).

DURATION

---

5 minutes

DATA OUTPUT DICTIONARY

The fields in the data files are:

(1) Raw data file: 'iowagamblingtask_raw*.iqdat'

build: The specific Inquisit version used (the 'build') that was run

computer.platform: the platform the script was run on (win/mac/ios/android)

date, time: date and time script was run

subject, group, with the current subject/groupnumber

session: with the current session id

deckOrderGroup: the experimental deck group (1-4) the subject was randomly assigned to

(refers to order of the advantageous (deck3, deck4) and disadvantegeous (deck 1, deck2)

decks on the screen)

group1: deck1(D), deck2(D), deck3(A), deck4(A)

group2: deck3(A), deck1(D), deck4(A), deck2(D)

group3: deck4(A), deck3(A), deck2(D), deck1(D)

group4: deck2(D), deck4(A), deck1(D), deck3(A)

blockCode, blockNum: the name and number of the current block (built-in Inquisit variable)

trialCode, trialNum: the name and number of the currently recorded trial (built-in Inquisit variable)

Note: trialNum is a built-in Inquisit variable; it counts all trials run;

even those that do not store data to the data file such

as feedback trials

cardsSelected: counts the number of card selections (across all blocks)

card1ItemNumber-
card2ItemNumber: store the itemnumber of the card face that needs to be presented on screen for each of the 4 decks

response: the participant's response

(e.g. which deck was chosen; the response "deck1" refers to the actual disadvantegous deck1

and not the position of the deck on the screen)

responseCategory: "advantageous" vs. "disadvantageous"
latency: the response latency (in ms)
gain: stores the trial's gain depending on choice
loss: stores the trial's loss depending on choice
currentTotal: stores the current amount of coins won

(2) Summary data file: 'iowagamblingtask_summary*.iqdat'

inquisit.version: Inquisit version run
computer.platform: the platform the script was run on (win/mac/ios/android)
startDate: date script was run
startTime: time script was started
subjectId: assigned subject id number
groupId: assigned group id number
sessionId: assigned session id number
elapsedTime: time it took to run script (in ms); measured from onset to offset of script
completed: 0 = script was not completed (prematurely aborted);
1 = script was completed (all conditions run)

deckOrderGroup: the experimental deck group (1-4) the subject was randomly assigned to
(refers to order of the advantageous (deck3, deck4) and disadvantegeous (deck 1, deck2) decks on the screen)
group1: deck1(D), deck2(D), deck3(A), deck4(A)
group2: deck3(A), deck1(D), deck4(A), deck2(D)
group3: deck4(A), deck3(A), deck2(D), deck1(D)
group4: deck2(D), deck4(A), deck1(D), deck3(A)

countDeck1Select-

countDeck4Select: count all the actual deck1-deck4 choices across all blocks

propDeck1Select-propDeck4Selectt: proportion of specific deck selections
Note: deck1 always refers to deck1 and not specific deck location

countAdvantageTotal: counts the total number of advantageous selections (deck 3 & deck 4)
countDisadvantageTotal: counts the total number of disadvantageous selections (deck 1 & deck 2)

net: difference btw. total number of advantageous and disadvantageous selections
(positive NET -> overall more advantageous selections)

per blocks of 20 trials:
countAdvantage1: counts the number of advantageous selections (deck 3 & deck 4) during trials 1-20
countDisadvantage1: counts the number of disadvantageous selections (deck 1 & deck 2) during trials 1-20
net1: difference btw. number of advantageous and disadvantageous selections during trials 1-20

countAdvantage2: counts the number of advantageous selections (deck 3 & deck 4) during trials 21-40
countDisadvantage2: counts the number of disadvantageous selections (deck 1 & deck 2) during trials 21-40
net2: difference btw. number of advantageous and disadvantageous selections during trials 21-40

countAdvantage3: counts the number of advantageous selections (deck 3 & deck 4) during trials 41-60
countDisadvantage3: counts the number of disadvantageous selections (deck 1 & deck 2) during trials 41-60
net3: difference btw. number of advantageous and disadvantageous selections during trials 41-60

countAdvantage4: counts the number of advantageous selections (deck 3 & deck 4) during trials 61-80
countDisadvantage4: counts the number of disadvantageous selections (deck 1 & deck 2) during trials 61-80

net4: difference btw. number of advantageous and disadvantageous selections during trials 61-80

countAdvantage5: counts the number of advantageous selections (deck 3 & deck 4) during trials 81-100
countDisadvantage5: counts the number of disadvantageous selections (deck 1 & deck 2) during trials 81-100
net5: difference btw. number of advantageous and disadvantageous selections during trials 81-100

EXPERIMENTAL SET-UP

---

* The game is set up to play 1 block of 100 trials
* The four decks are assigned a (permanent) screen location by groupnumber (4 different groups).
regardless of actual screen presentation:
deck1/deck2 = disadvantageous
deck3/deck4 = advantageous
* By default, in this script the win/loss probabilities assigned are aligned with the model of "once played,
cards are NOT returned to the deck" (therefore turning probabilities into actual frequencies over the course of 10 selections of each deck;
see more info under section EXPRESSIONS)
* the script collects performance data in blocks of 20 trials

---

STIMULI

---

provided by Millisecond - can be edited under section Editable Stimuli

---

INSTRUCTIONS

---

provided by Millisecond - can be edited under section Editable Instructions

---

EDITABLE CODE

---

check below for (relatively) easily editable parameters, stimuli, instructions etc.
Keep in mind that you can use this script as a template and therefore always "mess" with the entire code
to further customize your experiment.

The parameters you can change are:

/ startamount: the amount of money owned at the start of the game; startamount (default) = 2000
/showCardInfo: shows information about number of cards played/how many more to come

</userManual>

****
EDITABLE PARAMETERS: change editable parameters here
****

<parameters>
/ startAmount = 2000
/ showCardInfo = true
</parameters>

****
EDITABLE INSTRUCTIONS: change instructions here
****

****
****

!!!REMAINING CODE: Customize after careful consideration only!!!

****
****

****
****

****
****

DEFAULTS

****
****

****
****

script requires Inquisit "7.0.0.0" or higher

<defaults>
/ canvasAspectRatio = (4,3)
/ minimumVersion = "7.0.0.0"
/ inputDevice = mouse
/ fontStyle = ("Arial", 3%, false, false, false, false, 5, 0)
/ txBGColor = white
/ txColor = (0, 0, 0)
</defaults>

****
****

****
****

DATA

****
****

****
****

Note: data file explanations under User Manual Information at the top

****
raw data
****

<data>
/ columns = (build, computer.platform, date, time, subject, group, session, blockCode, blockNum,
trialCode, trialNum,
values.deckOrderGroup, values.cardsSelected, response, values.responseCategory, latency,
values.gain, values.loss, values.currentTotal)
</data>

****
summary data
****

<summaryData>
/ columns = (inquisit.version, computer.platform, script.startDate, script.startTime,
script.subjectId, script.groupId, script.sessionId, script.elapsedTime,
script.completed,
values.deckOrderGroup, values.currentTotal,
values.countAdvantageTotal, values.countDisadvantageTotal, expressions.net,
values.countAdvantage1, values.countDisadvantage1, expressions.net1,
values.countAdvantage2, values.countDisadvantage2, expressions.net2,
values.countAdvantage3, values.countDisadvantage3, expressions.net3,
values.countAdvantage4, values.countDisadvantage4, expressions.net4,
values.countAdvantage5, values.countDisadvantage5, expressions.net5)
</summaryData>
****
****
****
****
VALUES: automatically updated
****
****
****
****

previoustotal: the previous amount of money owned
cardselected: adds the number of cards selected
gain: stores the trial's gain depending on choice
loss: stores the trial's loss depending on choice

groupnumber: the groupnumber (1-4) the subject was randomly assigned to
currenttotal: the money currently won

responseCategory: "advantageous" vs. "disadvantageous"
/countAdvantage: counts the number of advantageous selections (deck 1 & deck 2)
/countDisadvantage: counts the number of disadvantageous selections (deck 3 & deck 4)

<values>
/ previousTotal = 0
/ cardsSelected = 0
/ gain = 0
/ loss = 0
/ deckOrderGroup = 0
/ currentTotal = 0
/ responseCategory = ""
/ countAdvantage1 = 0
/ countDisadvantage1 = 0
/ countAdvantage2 = 0
/ countDisadvantage2 = 0
/ countAdvantage3 = 0
/ countDisadvantage3 = 0
/ countAdvantage4 = 0
/ countDisadvantage4 = 0
/ countAdvantage5 = 0
/ countDisadvantage5 = 0
/ countAdvantageTotal = 0
/ countDisadvantageTotal = 0
</values>

****
****
****
****
EXPRESSIONS
****
****
****
****
/ getlossdeck1 -
/ getlossdeck4: calculates the current loss for a played deck

losses are selected randomly without replacement

=> across 10 trials per deck, you will see the following losses:

deck1/deck2 = overall disadvantageous
deck3/deck4 = overall advantageous

```
<list lossDeck1>
/ items = (0, 0, 0, 0, 0, 150, 200, 250, 300, 350)
/ replace = false
/ selectionRate = always
</list>

<list lossDeck2>
/ items = (0, 0, 0, 0, 0, 0, 0, 0, 0, 1250)
/ replace = false
/ selectionRate = always
</list>

<list lossDeck3>
/ items = (0, 0, 0, 0, 0, 50, 50, 50, 50, 50)
/ replace = false
/ selectionRate = always
</list>

<list lossDeck4>
/ items = (0, 0, 0, 0, 0, 0, 0, 0, 0, 250)
/ replace = false
/ selectionRate = always
</list>

<expressions>
/ getLossDeck1 = return list.lossDeck1.nextValue
/ getLossDeck2 = return list.lossDeck2.nextValue
/ getLossDeck3 = return list.lossDeck3.nextValue
/ getLossDeck4 = return list.lossDeck4.nextValue

/ net = return (values.countAdvantageTotal - values.countDisadvantageTotal)
/ net1 = return (values.countAdvantage1 - values.countDisadvantage1)
/ net2 = return (values.countAdvantage2 - values.countDisadvantage2)
/ net3 = return (values.countAdvantage3 - values.countDisadvantage3)
/ net4 = return (values.countAdvantage4 - values.countDisadvantage4)
/ net5 = return (values.countAdvantage5 - values.countDisadvantage5)
```

```
</expressions>

**********
**********
    INSTRUCTIONS
**********
**********

<text igtInstructions>
/ items = instructions
/ select = 0
/ position = (50%, 60%)
/ fontStyle = ("Arial", 1.75%)
/ size = (80%, 30%)
/ hJustify = left
/ vAlign = top
</text>

<trial instructions>
/ stimulusFrames = [1=igt, deck1, deck2, deck3, deck4, deck1Label, deck2Label,
deck3Label, deck4Label, igtInstructions, start]
/ validResponse = (start)
/ recordData = false
</trial>

<block instructions>
/ trials = [1=instructions]
</block>

**********
**********
    STIMULI
**********
**********

<text selectInstructions>
```

```
/ items = instructions
/ select = 1
/ position = (50%, 65%)
</text>

<picture deck1>
/ items = ("deck.jpg")
/ position = (20%, 50%)
/ vAlign = bottom
/ erase = false
/ size = (20%, 20%)
</picture>

<picture deck2>
/ items = ("deck.jpg")
/ position = (40%, 50%)
/ vAlign = bottom
/ erase = false
/ size = (20%, 20%)
</picture>

<picture deck3>
/ items = ("deck.jpg")
/ position = (60%, 50%)
/ vAlign = bottom
/ erase = false
/ size = (20%, 20%)
</picture>

<picture deck4>
/ items = ("deck.jpg")
/ position = (80%, 50%)
/ vAlign = bottom
/ erase = false
/ size = (20%, 20%)
</picture>

<picture deckon>
/ items = ("deckon.jpg")
/ erase = false
</picture>

**********
```

```
**********************************************************************************************
**********
    TRIALS
**********************************************************************************************
**********
**********************************************************************************************
**********

<trial igt>
/ onTrialBegin = {
    picture.deck1.setItem("deck.jpg", 0);
    picture.deck2.setItem("deck.jpg", 0);
    picture.deck3.setItem("deck.jpg", 0);
    picture.deck4.setItem("deck.jpg", 0);
}
/ stimulusFrames = [1=deck1, deck2, deck3, deck4, deck1Label, deck2Label,
deck3Label, deck4Label, selectInstructions]
/ validResponse = (deck1, deck2, deck3, deck4)

/ onTrialEnd = {
    values.cardsSelected++;

    if (this.response == "deck1") {
        picture.deck1.setItem("deckon.jpg", 0);
        text.gain.x=picture.deck1.x;
        text.loss.x=picture.deck1.x;
        values.gain=100;
        values.loss=expressions.getLossDeck1;
        values.previousTotal=values.currentTotal;
        values.currentTotal = values.currentTotal+values.gain-values.loss;
        values.responseCategory="disadvantageous";
        values.countDisadvantageTotal++;
        if (this.trialCount <= 20) {
                values.countDisadvantage1++;
        } else if (this.trialCount <= 40) {
                values.countDisadvantage2++;
        } else if (this.trialCount <= 60) {
                values.countDisadvantage3++;
        } else if (this.trialCount <= 80) {
                values.countDisadvantage4++;
        } else {
                values.countDisadvantage5++;
        }
    } else if (this.response == "deck2") {
```

```
        picture.deck2.setItem("deckon.jpg", 0);
        text.gain.x=picture.deck2.x;
        text.loss.x=picture.deck2.x;
        values.gain=100;
        values.loss=expressions.getLossDeck2;
        values.previousTotal=values.currentTotal;
        values.currentTotal = values.currentTotal+values.gain-values.loss;
        values.responseCategory="disadvantageous";
        values.countDisadvantageTotal++;
        if (this.trialCount <= 20) {
                values.countDisadvantage1++;
        } else if (this.trialCount <= 40) {
                values.countDisadvantage2++;
        } else if (this.trialCount <= 60) {
                values.countDisadvantage3++;
        } else if (this.trialCount <= 80) {
                values.countDisadvantage4++;
        } else {
                values.countDisadvantage5++;
        }
    } else if (this.response == "deck3") {
        picture.deck3.setItem("deckon.jpg", 0);
        text.gain.x=picture.deck3.x;
        text.loss.x=picture.deck3.x;
        values.gain=50;
        values.loss=expressions.getLossDeck3;
        values.previousTotal=values.currentTotal;
        values.currentTotal = values.currentTotal+values.gain-values.loss;
        values.responseCategory="advantageous";
        values.countAdvantageTotal++;
        if (this.trialCount <= 20) {
                values.countAdvantage1++;
        } else if (this.trialCount <= 40) {
                values.countAdvantage2++;
        } else if (this.trialCount <= 60) {
                values.countAdvantage3++;
        } else if (this.trialCount <= 80) {
                values.countAdvantage4++;
        } else {
                values.countAdvantage5++;
        }
    } else if (this.response == "deck4") {
        picture.deck4.setItem("deckon.jpg", 0);
        text.gain.x=picture.deck4.x;
```

```
        text.loss.x=picture.deck4.x;
        values.gain=50;
        values.loss=expressions.getLossDeck4;
        values.previousTotal=values.currentTotal;
        values.currentTotal = values.currentTotal+values.gain-values.loss;
        values.responseCategory="advantageous";
        values.countAdvantageTotal++;
        if (this.trialCount <= 20) {
                values.countAdvantage1++;
        } else if (this.trialCount <= 40) {
                values.countAdvantage2++;
        } else if (this.trialCount <= 60) {
                values.countAdvantage3++;
        } else if (this.trialCount <= 80) {
                values.countAdvantage4++;
        } else {
                values.countAdvantage5++;
        }
    }
}
/ branch = {
        return trial.igtFeedback;
}
</trial>

<trial igtFeedback>
/ stimulusFrames = [1=deck1, deck2, deck3, deck4, gain, loss, previousTotal,
currentTotal, cardsSelected, continue]
/ validResponse = (continue)
/ recordData = false
</trial>

*****************************************************************************************************
**********
*****************************************************************************************************
**********
    BLOCKS
*****************************************************************************************************
**********
*****************************************************************************************************
**********

<block igt>
/ onBlockBegin = {
```

```
    if ( values.deckOrderGroup == 1) {
        picture.deck1.x = 20%;
        picture.deck2.x = 40%;
        picture.deck3.x = 60%;
        picture.deck4.x = 80%;
    } else if ( values.deckOrderGroup == 2) {
        picture.deck1.x = 40%;
        picture.deck2.x = 80%;
        picture.deck3.x = 20%;
        picture.deck4.x = 60%;
    } else if ( values.deckOrderGroup == 3) {
        picture.deck1.x = 80%;
        picture.deck2.x = 60%;
        picture.deck3.x = 40%;
        picture.deck4.x = 20%;
    } else if ( values.deckOrderGroup == 4) {
        picture.deck1.x = 60%;
        picture.deck2.x = 20%;
        picture.deck3.x = 80%;
        picture.deck4.x = 40%;
    }
}
/ trials = [1-100 = igt]
</block>

**********************************************************************************************
**********
**********************************************************************************************
**********
    EXPERIMENT
**********************************************************************************************
**********
**********************************************************************************************
**********

<expt a>
/ groups = (1 of 4)
/ onExptBegin = {
    values.deckOrderGroup = 1;
    if (parameters.showCardInfo == false) {
        text.cardsSelected.textColor = white;
    }
    values.currentTotal = parameters.startAmount;
}
```

```
/ blocks = [
    1=instructions;
    2=igt;
]
</expt>

<expt b>
/ groups = (2 of 4)
/ onExptBegin = {
    values.deckOrderGroup = 2;
    if (parameters.showCardInfo == false) {
        text.cardsSelected.textColor = white;
    }
    values.currentTotal = parameters.startAmount;
}
/ blocks = [
    1=instructions;
    2=igt;
]
</expt>

<expt c>
/ groups = (3 of 4)
/ onExptBegin = {
    values.deckOrderGroup = 3;
    if (parameters.showCardInfo == false) {
        text.cardsSelected.textColor = white;
    }
    values.currentTotal = parameters.startAmount;
}
/ blocks = [
    1=instructions;
    2=igt;
]
</expt>

<expt d>
/ groups = (4 of 4)
/ onExptBegin = {
    values.deckOrderGroup = 4;
    if (parameters.showCardInfo == false) {
        text.cardsSelected.textColor = white;
    }
    values.currentTotal = parameters.startAmount;
```

```
}
/ blocks = [
    1=instructions;
    2=igt;
]
</expt>
```

****
****

End of File